

AGL Lakehouse v2

Enterprise-Scale Orchestration Guide — Fleet, Cold-Chain, Driver & Delivery Intelligence

This is the version-2 build-out of the AGL Lakehouse. Version 1 (the beginner + orchestration guides) proved the medallion pattern — Bronze → Silver → Gold, a Neon incremental sync, Auto Loader file ingestion, and one scheduled Job — on a small 5-table demo (`customers` , `warehouse_inventory` , `shipment_legs` , `container_telemetry` , `edi_invoice`).

Version 2 keeps every pattern from v1 (Secrets, watermark MERGE, delete reconciliation, Auto Loader with `trigger(availableNow=True)` , quarantine tables, the XML manifest fallback) and applies it, config-driven, across the **six real data domains** your Python generators produced: Shipment & Master Data, Fleet & Telematics, Driver Monitoring, Cold-Chain, Delivery & Milestones, and External/Market Data — **31 tables** in total. It also adds two new layers v1 didn't need: a **curated Silver layer** that joins domains together, and a **Gold feature/BI layer** purpose-built for the two audiences who will actually consume this lakehouse: Data Scientists and Business Leaders.

Setting	Value used in this guide
Catalog	<code>workspace</code> (same as v1)
New raw schema (Neon-synced masters + landing volume)	<code>agl_core</code>
New volume (file drops for all 6 domains)	<code>/Volumes/workspace/agl_core/agl_volume_v2/</code>
New medallion schemas	<code>bronze_v2</code> , <code>silver_v2</code> , <code>gold_v2</code>
Neon secret scope	<code>agl_neon</code> (reused from v1 — same Neon project, new tables)
Neon-synced master tables (8)	<code>customers</code> , <code>shippers</code> , <code>consignees</code> , <code>vehicles</code> , <code>drivers</code> , <code>containers</code> , <code>warehouses</code> , <code>sensors</code>
File-based Bronze sources (23)	see Part 2.3 — CSV, JSON, Parquet and one XML source
New notebooks (6)	<code>00_setup_schemas_v2</code> , <code>01_neon_master_sync_v2</code> , <code>02_bronze_file_ingestion_v2</code> , <code>03_silver_transformation_v2</code> , <code>04_gold_feature_store_v2</code> , <code>05_gold_business_views_v2</code>
Job name this guide creates	<code>AGL_ingestion_v2</code>

PREREQUISITE

Complete the beginner guide and the v1 Orchestration & Scheduling Guide first, and have `databricks_v1_proj` running. This guide does not touch or rename v1's objects — `agl_schema` , `bronze.dim_customer` , etc. are left exactly as they are. Everything here is additive, in its own schemas.

TABLE OF CONTENTS

Part 0 — Why v2? Scale, Domains & Design Decisions

3

Part 1 — Provision Neon for the 8 New Master Tables

4

Part 2 — Stage the Six Domains in the Volume

6

Part 3 — Notebook: 00_setup_schemas_v2

8

Part 4 — Notebook: 01_neon_master_sync_v2

9

Part 5 — Notebook: 02_bronze_file_ingestion_v2

11

Part 6 — Notebook: 03_silver_transformation_v2

14

Part 7 — Notebook: 04_gold_feature_store_v2 (Data Scientist tables)

18

Part 8 — Notebook: 05_gold_business_views_v2 (Business Leader views)

21

Part 9 — Orchestrate: the AGL_ingestion_v2 Job

23

Part 10 — Schedule, Scale & Performance

25

Part 11 — How Each Persona Actually Uses This

26

Part 12 — Troubleshooting (v2-specific)

28

Part 13 — Glossary Additions

29

PART 0

Why v2? Scale, Domains & Design Decisions

0.1 What's different from v1

v1 was one notebook syncing 3 Neon dimension tables and reading 5 file sources by hand-writing one Auto Loader cell per source. v2 has **31 tables across 6 domains** — writing 23 near-identical Auto Loader cells would be unreadable and impossible to maintain. So v2's biggest structural change is **config-driven ingestion**: every domain's sources are described as rows in a Python list (name, format, file glob, target table), and one loop/function ingests all of them. Adding a 24th source later means adding one row, not one more cell.

v2 also splits v1's single "sync + files" notebook into two independent notebooks (Neon sync, file ingestion) so they can run as **parallel Job tasks** instead of one long serial cell block, and it adds a **curated Silver layer** that joins domains together (trip + GPS + vehicle, sensor + container, weather + traffic by corridor/day) so the Gold layer doesn't have to repeat multi-way joins in every feature table.

0.2 The six domains at a glance

Domain	Source folder (local)	Tables	Destination
Shipment & Master Data	agl_data_output/	4 — 1 fact (agl_shipment_data), 3 master (customers, shippers, consignees)	3 masters → Neon · 1 fact → Bronze
Fleet & Telematics	agl_fleet_data_output/	5 — 1 master (vehicles), 4 fact (trips, gps_tracking, telematics, maintenance_logs)	vehicles → Neon · rest → Bronze
Driver Monitoring	agl_driver_data_output/	6 — 1 master (drivers), 5 fact (behavioral_events, app_telematics, driver_logs, pre_trip_inspections, fatigue_data)	drivers → Neon · rest → Bronze
Cold-Chain	agl_coldchain_data_output/	6 — 3 master (containers, warehouses, sensors), 3 fact (sensor_readings, reefer_status, sensor_health)	3 masters → Neon · rest → Bronze
Delivery & Milestones	agl_delivery_data_output/	5 — all event/fact (milestones, proof_of_delivery, exceptions, osd_logs, delivery_metrics)	all → Bronze
External / Market	agl_external_data_output/	5 — all context (weather_data, traffic_data, fuel_prices, exchange_rates, market_data)	all → Bronze

Total: 8 tables to Neon, 23 tables through Auto Loader/Volume ingestion = 31 tables.

0.3 Deciding what goes to Neon vs. the Volume

The v1 guide's rule (Part 0.2/0.3) still applies: Neon is for **mutable, bounded-cardinality entities** that get updated and occasionally deleted at the source and need a MERGE; the Volume + Auto Loader is for **append-only events and telemetry** that only ever grow. Applying that test across all 31 tables:

Signal	→ Neon (relational, synced)	→ Volume (files, Auto Loader)
Row count	Small — tens to a few hundred rows (vehicles: 50, containers: 100, drivers: 200, customers: 400)	Large and growing — thousands to millions of rows over time
Row lifecycle	Same row updated repeatedly (vehicle odometer, driver rating, container status) or removed (decommissioned asset)	Row is written once and never changes (a GPS ping, a sensor reading, a weather observation)
Real-world analogue	Fleet/HR/WMS master data living in an operational Postgres database	IoT telemetry streams, event logs, third-party API pulls, EDI/OS&D documents
Result in this guide	customers, shippers, consignees, vehicles, drivers, containers, warehouses, sensors	everything else — shipments, trips, GPS, telematics, maintenance logs, all driver-monitoring streams, all cold-chain telemetry, all delivery/milestone events, all external/market feeds

NOTE

Two tables look borderline. `maintenance_logs` reads like it could be relational, but each row is a closed, immutable service event (a work order once completed never changes) — that's an event log, so it stays in Bronze. `sensors` looks like a telemetry source, but the table itself is a small device *registry* (battery %, calibration date, status) that gets updated in place as the device ages — that's a master table, so it goes to Neon. When unsure, ask: "does this row get UPDATED after it's created?" Yes → Neon. No → Volume.

0.4 Medallion map for v2

PART 1

Provision Neon for the 8 New Master Tables

1.1 Reuse your existing secret scope

You already created the `agl_neon` secret scope and its `host/port/database/username/password` keys in the v1 guide (Part 1). These new tables live in the **same Neon Postgres database**, so there is nothing new to create here — `00b_agl_ingestion`'s connection cell pattern is reused as-is in Part 4 below. If you're pointing v2 at a different Neon project, repeat v1 Part 1.1–1.2 with scope name `agl_neon_v2` instead.

1.2 Create the 8 tables in Neon

Run this once against your Neon database (Neon SQL Editor, or `psql`). Every table gets an `updated_at` column plus a trigger that stamps it automatically on every `UPDATE` — this is what makes the watermark-based incremental sync from v1 Part 2 work efficiently instead of falling back to a full-table compare.

```
-- shared trigger function, create once
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- 1. customers
CREATE TABLE customers (
    customer_id    VARCHAR(20) PRIMARY KEY,
    customer_name  TEXT,
    customer_type  VARCHAR(50),
    credit_limit   NUMERIC(14,2),
    payment_terms  VARCHAR(30),
    address        TEXT,
    contact_email  VARCHAR(255),
    contact_phone  VARCHAR(50),
    updated_at     TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_customers_updated_at BEFORE UPDATE ON customers
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 2. shippers
CREATE TABLE shippers (
    shipper_id     VARCHAR(20) PRIMARY KEY,
    shipper_name   TEXT,
    address        TEXT,
    contact_person  VARCHAR(120),
    contact_email  VARCHAR(255),
    contact_phone  VARCHAR(50),
    updated_at     TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_shippers_updated_at BEFORE UPDATE ON shippers
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();
```

```
-- 3. consignees
CREATE TABLE consignees (
  consignee_id    VARCHAR(20) PRIMARY KEY,
  consignee_name  TEXT,
  address         TEXT,
  contact_person  VARCHAR(120),
  contact_email   VARCHAR(255),
  contact_phone   VARCHAR(50),
  updated_at      TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_consignees_updated_at BEFORE UPDATE ON consignees
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 4. vehicles
CREATE TABLE vehicles (
  vehicle_id      VARCHAR(20) PRIMARY KEY,
  license_plate    VARCHAR(20),
  model           VARCHAR(100),
  year            INT,
  fuel_capacity_l  NUMERIC(10,2),
  avg_consumption_l_per_100km NUMERIC(10,2),
  current_fuel_level NUMERIC(10,2),
  odometer_reading_km NUMERIC(12,2),
  truck_type      VARCHAR(30),
  max_load_tonnes NUMERIC(10,2),
  status          VARCHAR(30),
  assigned_driver  VARCHAR(120),
  assigned_corridor VARCHAR(100),
  insurance_expiry TIMESTAMP,
  last_inspection_date TIMESTAMP,
  next_inspection_date TIMESTAMP,
  updated_at      TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_vehicles_updated_at BEFORE UPDATE ON vehicles
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 5. drivers
CREATE TABLE drivers (
  driver_id      VARCHAR(20) PRIMARY KEY,
  first_name     VARCHAR(80),
  last_name      VARCHAR(80),
  email          VARCHAR(255),
  phone          VARCHAR(50),
  license_number  VARCHAR(30),
  license_class   VARCHAR(10),
  license_expiry  TIMESTAMP,
  date_of_birth  TIMESTAMP,
  hire_date      TIMESTAMP,
  status         VARCHAR(30),
  vehicle_type    VARCHAR(30),
  home_terminal   VARCHAR(100),
  emergency_contact VARCHAR(120),
  emergency_phone VARCHAR(50),
  total_experience_years NUMERIC(5,2),
  last_medical_exam  TIMESTAMP,
  next_medical_exam  TIMESTAMP,
  preferred_language VARCHAR(30),
  rating          NUMERIC(3,2),
  total_missions    INT,
  total_km_driven    NUMERIC(12,2),
  updated_at        TIMESTAMP NOT NULL DEFAULT now()
);
```

```

);
CREATE TRIGGER trg_drivers_updated_at BEFORE UPDATE ON drivers
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 6. containers
CREATE TABLE containers (
  container_id          VARCHAR(20) PRIMARY KEY,
  container_type        VARCHAR(20),
  product_type         VARCHAR(60),
  setpoint_temp_c      NUMERIC(6,2),
  temp_min_c           NUMERIC(6,2),
  temp_max_c           NUMERIC(6,2),
  setpoint_humidity_percent NUMERIC(6,2),
  humidity_min_percent NUMERIC(6,2),
  humidity_max_percent NUMERIC(6,2),
  capacity_m3          NUMERIC(10,2),
  current_fill_percent NUMERIC(6,2),
  manufacturer         VARCHAR(80),
  model                VARCHAR(80),
  year_manufactured    INT,
  last_maintenance_date TIMESTAMP,
  next_maintenance_date TIMESTAMP,
  status               VARCHAR(30),
  assigned_warehouse   VARCHAR(100),
  gps_latitude         NUMERIC(10,6),
  gps_longitude        NUMERIC(10,6),
  updated_at           TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_containers_updated_at BEFORE UPDATE ON containers
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 7. warehouses
CREATE TABLE warehouses (
  warehouse_id          VARCHAR(20) PRIMARY KEY,
  warehouse_name        VARCHAR(150),
  address               TEXT,
  total_area_m2         NUMERIC(10,2),
  cooling_capacity_kw     NUMERIC(10,2),
  number_of_bays        INT,
  product_types         TEXT,
  temperature_zone      VARCHAR(60),
  power_backup_hours    NUMERIC(6,2),
  generator_capacity_kva NUMERIC(10,2),
  manager_name          VARCHAR(120),
  contact_phone         VARCHAR(50),
  contact_email         VARCHAR(255),
  operating_hours       VARCHAR(30),
  certification         VARCHAR(100),
  status                VARCHAR(30),
  updated_at            TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_warehouses_updated_at BEFORE UPDATE ON warehouses
  FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- 8. sensors
CREATE TABLE sensors (
  sensor_id             VARCHAR(20) PRIMARY KEY,
  sensor_type           VARCHAR(60),
  manufacturer          VARCHAR(80),
  model                VARCHAR(80),
  container_id          VARCHAR(20),
  warehouse_id          VARCHAR(20),

```

```

location          VARCHAR(60),
battery_level_percent NUMERIC(6,2),
signal_strength_dbm NUMERIC(6,2),
calibration_date   TIMESTAMP,
next_calibration_date TIMESTAMP,
calibration_interval_days INT,
status            VARCHAR(30),
firmware_version   VARCHAR(30),
last_reading       NUMERIC(10,2),
last_reading_timestamp TIMESTAMP,
updated_at         TIMESTAMP NOT NULL DEFAULT now()
);
CREATE TRIGGER trg_sensors_updated_at BEFORE UPDATE ON sensors
FOR EACH ROW EXECUTE FUNCTION set_updated_at();

```

Expected result: 8 tables + 8 triggers created, no errors. Run `\dt` in psql or check the Neon console's Tables tab to confirm all 8 appear.

1.3 Seed Neon from your local master-data files

Unlike v1 (where Neon already held live data), these 8 tables are empty until you load your generator output into them. Run this once from your local machine — **not** in a Databricks notebook — using Python:

```

pip install pandas sqlalchemy psycopg2-binary

# seed_neon_v2.py
import pandas as pd
from sqlalchemy import create_engine

NEON_HOST = "ep-xxxx.us-east-2.aws.neon.tech" # from your Neon console
NEON_PORT = 5432
NEON_DB = "neondb"
NEON_USER = "your_user"
NEON_PASS = "your_password"

engine = create_engine(
    f"postgresql+psycopg2://{NEON_USER}:{NEON_PASS}@{NEON_HOST}:{NEON_PORT}/{NEON_DB}?sslmode=require"
)

# (local_folder, filename, target Neon table)
seed_sources = [
    ("agl_data_output", "customers_master.csv", "customers"),
    ("agl_data_output", "shippers_master.csv", "shippers"),
    ("agl_data_output", "consignees_master.csv", "consignees"),
    ("agl_fleet_data_output", "vehicles.csv", "vehicles"),
    ("agl_driver_data_output", "drivers.csv", "drivers"),
    ("agl_coldchain_data_output", "containers.csv", "containers"),
    ("agl_coldchain_data_output", "warehouses.csv", "warehouses"),
    ("agl_coldchain_data_output", "sensors.csv", "sensors"),
]

for folder, filename, table in seed_sources:
    df = pd.read_csv(f"{folder}/{filename}")
    df.to_sql(table, engine, if_exists="append", index=False, method="multi", chunksize=500)
    print(f"Seeded {len(df)} rows into {table}")

```

Expected result: eight "Seeded N rows into <table>" lines. `to_sql` ignores the extra `updated_at` column (it isn't in the DataFrame), so Postgres fills it with `now()` from the column default — every seeded row starts with a real timestamp, ready for the watermark sync.

WATCH OUT

Run this script exactly once. Re-running it appends duplicate primary keys and the insert will fail on the primary-key constraint — that's expected and safe (it just means seeding already happened); truncate the tables first if you genuinely need to reseed.

1.4 Verify

```
SELECT 'customers' t, count(*) FROM customers UNION ALL
SELECT 'shippers', count(*) FROM shippers UNION ALL
SELECT 'consignees', count(*) FROM consignees UNION ALL
SELECT 'vehicles', count(*) FROM vehicles UNION ALL
SELECT 'drivers', count(*) FROM drivers UNION ALL
SELECT 'containers', count(*) FROM containers UNION ALL
SELECT 'warehouses', count(*) FROM warehouses UNION ALL
SELECT 'sensors', count(*) FROM sensors;
```

Expected result: 8 rows back, one count per table, all > 0 and matching the row counts of your local CSVs (customers ≈ 400, shippers ≈ 200, consignees ≈ 300, vehicles ≈ 50, drivers ≈ 200, containers ≈ 100, warehouses ≈ 20, sensors ≈ 246).

PART 2

Stage the Six Domains in the Volume

2.1 Create the new Volume

1. In Databricks, open **Catalog** → `workspace` → create schema `agl_core` if it doesn't exist yet (Part 3 also does this from a notebook, either works).
2. Inside `agl_core`, create a Volume named `agl_volume_v2`.
3. Inside the volume, create six folders matching the domains: `shipment/`, `fleet/`, `driver/`, `coldchain/`, `delivery/`, `external/`.

Expected result: `/Volumes/workspace/agl_core/agl_volume_v2/` exists with 6 empty subfolders.

2.2 Upload strategy

Each generator wrote every table in 4 formats (CSV/JSON/Parquet/XML) so you could choose. Upload **only the one format per source** listed in 2.3 below — don't upload all 4, Auto Loader would otherwise ingest the same rows twice under two different target tables. Using the Databricks CLI from your local machine:

```
# one example per domain – repeat with the right local file per Part 2.3's format column
databricks fs cp "agl_data_output/agl_shipment_data.csv"
/Volumes/workspace/agl_core/agl_volume_v2/shipment/
databricks fs cp "agl_fleet_data_output/gps_tracking.parquet"
/Volumes/workspace/agl_core/agl_volume_v2/fleet/
databricks fs cp "agl_driver_data_output/behavioral_events.json"
/Volumes/workspace/agl_core/agl_volume_v2/driver/
databricks fs cp "agl_coldchain_data_output/sensor_readings.parquet"
/Volumes/workspace/agl_core/agl_volume_v2/coldchain/
databricks fs cp "agl_delivery_data_output/proof_of_delivery.xml"
/Volumes/workspace/agl_core/agl_volume_v2/delivery/
databricks fs cp "agl_external_data_output/weather_data.json"
/Volumes/workspace/agl_core/agl_volume_v2/external/
```

Prefer clicking? The Catalog Explorer's volume page has an **Upload to this volume** button that does the same thing per-folder.

NOTE

When you regenerate fresh data later (a new day's run of your Python generators), upload it under a dated filename — e.g. `gps_tracking_20260801.parquet` — next to the original. The globs in Part 5 already match `<name>*.<ext>`, so Auto Loader will pick up the new file automatically on the next scheduled run without you touching any code, exactly like v1 Part 3.2's `customers*.csv` pattern.

2.3 Format assignment — the 23 file-based Bronze sources

Domain	Source table	Format to upload	Why
Shipment	<code>agl_shipment_data</code>	CSV	Large tabular fact table — CSV keeps it simple and human-inspectable

Fleet	gps_tracking	Parquet	Highest-frequency telemetry — Parquet is columnar and far cheaper to scan
	telematics	Parquet	Same — high-frequency engine/sensor stream
	trips	CSV	One row per trip — moderate volume, simple tabular shape
	maintenance_logs	CSV	Event log, moderate volume
Driver	behavioral_events	JSON	Many optional/nullable fields depending on event_type — JSON tolerates sparse schemas
	driver_logs	JSON	HOS duty-status stream, same sparse-field reasoning
	app_telematics	CSV	Regular session-shaped rows
	pre_trip_inspections	CSV	Regular checklist-shaped rows (one embedded JSON string column carried through as-is)
	fatigue_data	JSON	Field set varies heavily by fatigue_type (yawning vs. phone vs. drowsiness) — JSON's the natural fit
Cold-Chain	sensor_readings	Parquet	Highest-volume table in the whole dataset — Parquet for scan efficiency
	reefer_status	Parquet	High-frequency reefer telemetry
	sensor_health	CSV	Periodic health-check rows, moderate volume
Delivery	milestones	JSON	Sequential event stream per shipment
	exceptions	JSON	Sparse optional fields depending on exception_type
	osd_logs	CSV	Regular tabular shape
	proof_of_delivery	XML	Treated as a legal/document artifact (like v1's EDI invoice) — uses the manifest-table fallback pattern, not streaming Auto Loader
	delivery_metrics	CSV	One computed row per shipment
External	weather_data	JSON	Simulates a weather API response payload
	traffic_data	JSON	Simulates a traffic API response payload
	fuel_prices	CSV	Simple financial feed export
	exchange_rates	CSV	Simple financial feed export
	market_data	CSV	Simple financial feed export

PART 3

Notebook: 00_setup_schemas_v2

Create a notebook named `00_setup_schemas_v2`, attach it to your cluster, single cell:

```
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.agl_core")
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.bronze_v2")
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.silver_v2")
spark.sql("CREATE SCHEMA IF NOT EXISTS workspace.gold_v2")

spark.sql("""
CREATE VOLUME IF NOT EXISTS workspace.agl_core.agl_volume_v2
""")

for folder in ["shipment", "fleet", "driver", "coldchain", "delivery", "external", "_checkpoints"]:
    dbutils.fs.mkdirs(f"/Volumes/workspace/agl_core/agl_volume_v2/{folder}")

print("v2 schemas, volume and domain folders ready.")
```

Expected result: prints the confirmation line; `agl_core`, `bronze_v2`, `silver_v2`, `gold_v2` all appear in Catalog Explorer.

PART 4

Notebook: 01_neon_master_sync_v2

This is v1's `00b_agl_ingestion` Part A, generalized from 3 tables to 8. Same connection cell, same watermark control table, same MERGE + delete-reconciliation cells — just driven by a longer config list.

4.1 Connection cell (identical pattern to v1 Part 2.1)

```
catalog = "workspace"
core_schema = "agl_core"

host      = dbutils.secrets.get(scope="agl_neon", key="host")
port      = dbutils.secrets.get(scope="agl_neon", key="port")
database  = dbutils.secrets.get(scope="agl_neon", key="database")
username  = dbutils.secrets.get(scope="agl_neon", key="username")
password  = dbutils.secrets.get(scope="agl_neon", key="password")

jdbc_url = f"jdbc:postgresql://{host}:{port}/{database}?sslmode=require"
connection_props = {"user": username, "password": password, "driver": "org.postgresql.Driver"}
print("Neon connection configured for database:", database)
```

4.2 The 8-table config list

```
neon_sources = [
    {"neon_table": "customers", "target_table": f"{catalog}.{core_schema}.dim_customer",
     "primary_key": "customer_id", "watermark_col": "updated_at"},
    {"neon_table": "shippers", "target_table": f"{catalog}.{core_schema}.dim_shipper",
     "primary_key": "shipper_id", "watermark_col": "updated_at"},
    {"neon_table": "consignees", "target_table": f"{catalog}.{core_schema}.dim_consignee",
     "primary_key": "consignee_id", "watermark_col": "updated_at"},
    {"neon_table": "vehicles", "target_table": f"{catalog}.{core_schema}.dim_vehicle",
     "primary_key": "vehicle_id", "watermark_col": "updated_at"},
    {"neon_table": "drivers", "target_table": f"{catalog}.{core_schema}.dim_driver",
     "primary_key": "driver_id", "watermark_col": "updated_at"},
    {"neon_table": "containers", "target_table": f"{catalog}.{core_schema}.dim_container",
     "primary_key": "container_id", "watermark_col": "updated_at"},
    {"neon_table": "warehouses", "target_table": f"{catalog}.{core_schema}.dim_warehouse",
     "primary_key": "warehouse_id", "watermark_col": "updated_at"},
    {"neon_table": "sensors", "target_table": f"{catalog}.{core_schema}.dim_sensor",
     "primary_key": "sensor_id", "watermark_col": "updated_at"},
]
```

4.3 Watermark control table (unchanged from v1 Part 2.3)

```
spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.{core_schema}._ingestion_watermarks (
    target_table STRING,
    last_watermark TIMESTAMP
)
""")
```

```
def get_last_watermark(target_table):
    row = spark.sql(f"""
        SELECT last_watermark FROM {catalog}.{core_schema}._ingestion_watermarks
        WHERE target_table = '{target_table}'
    """).collect()
    return row[0]["last_watermark"] if row else "1900-01-01 00:00:00"

def set_last_watermark(target_table, new_watermark):
    spark.sql(f"""
        MERGE INTO {catalog}.{core_schema}._ingestion_watermarks t
        USING (SELECT '{target_table}' AS target_table, TIMESTAMP('{new_watermark}') AS last_watermark)
        s
        ON t.target_table = s.target_table
        WHEN MATCHED THEN UPDATE SET t.last_watermark = s.last_watermark
        WHEN NOT MATCHED THEN INSERT (target_table, last_watermark) VALUES (s.target_table,
        s.last_watermark)
    """)

print("Watermark control table ready.")
```

4.4 Upsert loop — one pass over all 8 sources

```
for src in neon_sources:
    last_wm = get_last_watermark(src["target_table"])
    query = (
        f"(SELECT * FROM {src['neon_table']}) "
        f"WHERE {src['watermark_col']} > TIMESTAMP '{last_wm}' AS changed_rows"
    )
    df_changed = spark.read.jdbc(url=jdbc_url, table=query, properties=connection_props)
    change_count = df_changed.count()

    if change_count == 0:
        print(f"{src['target_table']}: no changes since {last_wm}")
        continue

    df_changed.createOrReplaceTempView("staged_changes")
    spark.sql(f"""
        MERGE INTO {src['target_table']} t
        USING staged_changes s
        ON t.{src['primary_key']} = s.{src['primary_key']}
        WHEN MATCHED THEN UPDATE SET *
        WHEN NOT MATCHED THEN INSERT *
    """)
    new_wm = df_changed.agg({src["watermark_col"]: "max"}).collect()[0][0]
    set_last_watermark(src["target_table"], new_wm)
    print(f"{src['target_table']}: merged {change_count} row(s), watermark advanced to {new_wm}")
```

Expected result: on the first run, 8 lines like "merged 400 row(s)" (every row looks "new" against the 1900-01-01 default watermark) — that's correct.

4.5 Delete reconciliation loop (unchanged pattern from v1 Part 2.5)

```
for src in neon_sources:
    df_live_keys = spark.read.jdbc(
        url=jdbc_url,
        table=f"(SELECT {src['primary_key']} FROM {src['neon_table']}) AS live_keys",
        properties=connection_props,
    )
```

```
df_live_keys.createOrReplaceTempView("live_keys")

deleted = spark.sql(f"""
    SELECT t.{src['primary_key']}
    FROM {src['target_table']} t
    LEFT ANTI JOIN live_keys l ON t.{src['primary_key']} = l.{src['primary_key']}
""")
n_deleted = deleted.count()
if n_deleted > 0:
    deleted.createOrReplaceTempView("deleted_keys")
    spark.sql(f"""
        DELETE FROM {src['target_table']} t
        WHERE EXISTS (SELECT 1 FROM deleted_keys d WHERE d.{src['primary_key']} = t.
{src['primary_key']})
        """)
    print(f"{src['target_table']}: removed {n_deleted} row(s) deleted at source")
else:
    print(f"{src['target_table']}: no deletes to reconcile")
```

WATCH OUT

All 8 master tables together are only ~1,500 rows, so the full-primary-key delete scan (Part 4.5) runs in under a second per table — no need for the logical-replication upgrade v1 mentions until these tables grow into the millions.

PART 5

Notebook: 02_bronze_file_ingestion_v2

One config-driven loop replaces 22 hand-written Auto Loader cells; a second, small function handles the one XML source.

5.1 Paths and the 22 CSV/JSON/Parquet sources

```
catalog = "workspace"
volume_path = "/Volumes/workspace/agl_core/agl_volume_v2"
checkpoint_base = f"{volume_path}/_checkpoints"
bronze_schema = "bronze_v2"

bronze_sources = [
    # -- shipment --
    {"domain": "shipment", "name": "agl_shipment_data", "format": "csv", "glob":
"shipment/agl_shipment_data*.csv", "target": "shipment_data"},
    # -- fleet --
    {"domain": "fleet", "name": "gps_tracking", "format": "parquet", "glob":
"fleet/gps_tracking*.parquet", "target": "fleet_gps_tracking"},
    {"domain": "fleet", "name": "telematics", "format": "parquet", "glob":
"fleet/telematics*.parquet", "target": "fleet_telematics"},
    {"domain": "fleet", "name": "trips", "format": "csv", "glob":
"fleet/trips*.csv", "target": "fleet_trips"},
    {"domain": "fleet", "name": "maintenance_logs", "format": "csv", "glob":
"fleet/maintenance_logs*.csv", "target": "fleet_maintenance_logs"},
    # -- driver --
    {"domain": "driver", "name": "behavioral_events", "format": "json", "glob":
"driver/behavioral_events*.json", "target": "driver_behavioral_events"},
    {"domain": "driver", "name": "driver_logs", "format": "json", "glob":
"driver/driver_logs*.json", "target": "driver_logs"},
    {"domain": "driver", "name": "app_telematics", "format": "csv", "glob":
"driver/app_telematics*.csv", "target": "driver_app_telematics"},
    {"domain": "driver", "name": "pre_trip_inspections", "format": "csv", "glob":
"driver/pre_trip_inspections*.csv", "target": "driver_pre_trip_inspections"},
    {"domain": "driver", "name": "fatigue_data", "format": "json", "glob":
"driver/fatigue_data*.json", "target": "driver_fatigue_data"},
    # -- coldchain --
    {"domain": "coldchain", "name": "sensor_readings", "format": "parquet", "glob":
"coldchain/sensor_readings*.parquet", "target": "coldchain_sensor_readings"},
    {"domain": "coldchain", "name": "reefer_status", "format": "parquet", "glob":
"coldchain/reefer_status*.parquet", "target": "coldchain_reefer_status"},
    {"domain": "coldchain", "name": "sensor_health", "format": "csv", "glob":
"coldchain/sensor_health*.csv", "target": "coldchain_sensor_health"},
    # -- delivery (XML source handled separately in 5.3) --
    {"domain": "delivery", "name": "milestones", "format": "json", "glob":
"delivery/milestones*.json", "target": "delivery_milestones"},
    {"domain": "delivery", "name": "exceptions", "format": "json", "glob":
"delivery/exceptions*.json", "target": "delivery_exceptions"},
    {"domain": "delivery", "name": "osd_logs", "format": "csv", "glob":
"delivery/osd_logs*.csv", "target": "delivery_osd_logs"},
    {"domain": "delivery", "name": "delivery_metrics", "format": "csv", "glob":
"delivery/delivery_metrics*.csv", "target": "delivery_metrics"},
    # -- external --
```



```

    {"domain": "external", "name": "weather_data", "format": "json", "glob":
"external/weather_data*.json", "target": "ext_weather_data"},
    {"domain": "external", "name": "traffic_data", "format": "json", "glob":
"external/traffic_data*.json", "target": "ext_traffic_data"},
    {"domain": "external", "name": "fuel_prices", "format": "csv", "glob":
"external/fuel_prices*.csv", "target": "ext_fuel_prices"},
    {"domain": "external", "name": "exchange_rates", "format": "csv", "glob":
"external/exchange_rates*.csv", "target": "ext_exchange_rates"},
    {"domain": "external", "name": "market_data", "format": "csv", "glob":
"external/market_data*.csv", "target": "ext_market_data"},
]
print(f"{len(bronze_sources)} file-based sources configured.")

```

5.2 One ingestion function, one loop

```

def ingest_source(src):
    reader = (
        spark.readStream.format("cloudFiles")
        .option("cloudFiles.format", src["format"])
        .option("cloudFiles.schemaLocation", f"{checkpoint_base}/{src['target']}_{schema}")
        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
    )
    if src["format"] == "csv":
        reader = reader.option("header", "true")
    if src["format"] == "json":
        reader = reader.option("multiline", "true")

    df = reader.load(f"{volume_path}/{src['glob']}")

    (df.writeStream
     .format("delta")
     .option("checkpointLocation", f"{checkpoint_base}/{src['target']}_ckpt")
     .option("mergeSchema", "true")
     .outputMode("append")
     .trigger(availableNow=True)
     .toTable(f"{catalog}.{bronze_schema}.{src['target']}")
     .awaitTermination())

results = []
for src in bronze_sources:
    try:
        ingest_source(src)
        results.append((src["target"], "OK"))
        print(f"{src['target']}: batch complete.")
    except Exception as e:
        results.append((src["target"], f"FAILED: {e}"))
        print(f"{src['target']}: FAILED - {e}")

failures = [r for r in results if r[1] != "OK"]
print(f"\n{len(results) - len(failures)}/{len(results)} sources ingested successfully.")
if failures:
    print("Failed sources:", failures)

```

Expected result: 22 "batch complete" lines, then "22/22 sources ingested successfully." The try/except means one bad source (e.g. a schema-drift error) doesn't abort the other 21 — this is important at v2's scale, where you don't want a single malformed file to fail the entire ingestion notebook.

TIP

Because every source lives in one list, adding a new file (e.g. a 7th driver-monitoring stream next quarter) means appending one dict to `bronze_sources` — no new cell, no copy-paste.

5.3 The one XML source: `proof_of_delivery` (manifest-table fallback)

Generalizing v1 Part 3.5's pattern (Auto Loader doesn't support XML) to a reusable function:

```
from pyspark.sql.functions import col

spark.sql(f"""
CREATE TABLE IF NOT EXISTS {catalog}.agl_core._processed_xml_files (
    file_path STRING,
    target_table STRING,
    processed_at TIMESTAMP
)
""")

def ingest_xml_source(folder, row_tag, target_table, column_map):
    all_files = [f.path for f in dbutils.fs.ls(f"{volume_path}/{folder}") if f.name.endswith(".xml")]
    already = set(r["file_path"] for r in spark.sql(f"""
        SELECT file_path FROM {catalog}.agl_core._processed_xml_files
        WHERE target_table = '{target_table}'
    """).collect())
    new_files = [f for f in all_files if f not in already]

    if not new_files:
        print(f"{target_table}: no new XML files to process.")
        return

    df_raw = spark.read.format("xml").option("rowTag", row_tag).load(new_files)
    df_clean = df_raw.select(*[col(src).alias(dst) for src, dst in column_map.items()])
    df_clean.write.mode("append").saveAsTable(f"{catalog}.{bronze_schema}.{target_table}")

    from pyspark.sql import Row
    from datetime import datetime
    spark.createDataFrame(
        [Row(file_path=f, target_table=target_table, processed_at=datetime.utcnow()) for f in new_files]
    ).write.mode("append").saveAsTable(f"{catalog}.agl_core._processed_xml_files")
    print(f"{target_table}: processed {len(new_files)} new XML file(s).")

pod_column_map = {
    "podid": "pod_id", "shipmentid": "shipment_id", "recipientname": "recipient_name",
    "recipientcontact": "recipient_contact", "recipientemail": "recipient_email",
    "deliveryagentname": "delivery_agent_name", "deliverytimestamp": "delivery_timestamp",
    "deliverylocation": "delivery_location", "cargoconditionsummary": "cargo_condition_summary",
    "cargoconditiondetails": "cargo_condition_details", "photoevidence": "photo_evidence",
    "temperatureatdeliveryc": "temperature_at_delivery_c", "humidityatdeliverypercent":
    "humidity_at_delivery_percent",
    "deliverynotes": "delivery_notes", "receivedby": "received_by",
}
ingest_xml_source("delivery", "ProofOfDelivery", "delivery_proof_of_delivery", pod_column_map)
```

NOTE

The `column_map` keys use the flattened, lower-cased tag names your Python generator's XML exporter produces (matching the pattern seen in v1's `gps_tracking.xml` — e.g. `<recipientname>`, no underscores). Confirm the exact row tag and field names against your own `proof_of_delivery.xml` before running — open the file and check the first record, exactly as the beginner guide taught for EDI invoices.

PART 6

Notebook: 03_silver_transformation_v2

Two jobs happen here: (1) a generic cleanse pass over every Bronze table, and (2) five **curated, cross-domain** tables that the Gold layer will build on — this is the layer v1 didn't need, because v1 never had to join fleet data with weather data with driver data.

6.1 Generic cleanse loop

```
from pyspark.sql.functions import current_timestamp, to_date, col

cleanse_map = [
    # (bronze table, silver table, natural key columns for de-dup, date column to derive event_date
    from)
    ("shipment_data", "shipment_data", ["shipment_id"], "order_date"),
    ("fleet_gps_tracking", "fleet_gps_tracking", ["vehicle_id", "trip_id", "timestamp"], "timestamp"),
    ("fleet_telematics", "fleet_telematics", ["vehicle_id", "timestamp"], "timestamp"),
    ("fleet_trips", "fleet_trips", ["trip_id"], "start_time"),
    ("fleet_maintenance_logs", "fleet_maintenance_logs", ["maintenance_id"], "service_date"),
    ("driver_behavioral_events", "driver_behavioral_events", ["event_id"], "timestamp"),
    ("driver_logs", "driver_logs", ["log_id"], "start_time"),
    ("driver_app_telematics", "driver_app_telematics", ["session_id"], "login_time"),
    ("driver_pre_trip_inspections", "driver_pre_trip_inspections", ["inspection_id"], "inspection_date"),
    ("driver_fatigue_data", "driver_fatigue_data", ["fatigue_id"], "timestamp"),
    ("coldchain_sensor_readings", "coldchain_sensor_readings", ["reading_id"], "timestamp"),
    ("coldchain_reefer_status", "coldchain_reefer_status", ["status_id"], "timestamp"),
    ("coldchain_sensor_health", "coldchain_sensor_health", ["health_id"], "check_timestamp"),
    ("delivery_milestones", "delivery_milestones", ["shipment_id", "milestone_name", "timestamp"], "timestamp"),
    ("delivery_exceptions", "delivery_exceptions", ["exception_id"], "occurred_at"),
    ("delivery_osd_logs", "delivery_osd_logs", ["osd_id"], "reported_date"),
    ("delivery_metrics", "delivery_metrics", ["shipment_id"], "expected_delivery"),
    ("delivery_proof_of_delivery", "delivery_proof_of_delivery", ["pod_id"], "delivery_timestamp"),
    ("ext_weather_data", "ext_weather_data", ["weather_id"], "timestamp"),
    ("ext_traffic_data", "ext_traffic_data", ["traffic_id"], "timestamp"),
    ("ext_fuel_prices", "ext_fuel_prices", ["fuel_price_id"], "timestamp"),
    ("ext_exchange_rates", "ext_exchange_rates", ["exchange_rate_id"], "timestamp"),
    ("ext_market_data", "ext_market_data", ["market_id"], "timestamp"),
]

for bronze_tbl, silver_tbl, keys, date_col in cleanse_map:
    df = spark.table(f"workspace.bronze_v2.{bronze_tbl}")
    df_clean = (
        df.dropDuplicates(keys)
        .withColumn("event_date", to_date(col(date_col)))
    )
```

```

        .withColumn("_silver_loaded_at", current_timestamp())
    )
    df_clean.write.mode("overwrite").option("overwriteSchema", "true") \
        .partitionBy("event_date") \
        .saveAsTable(f"workspace.silver_v2.{silver_tbl}")
    print(f"silver_v2.{silver_tbl}: {df_clean.count()} rows (partitioned by event_date)")

```

Expected result: 23 lines, one per source, each reporting a row count. Every Silver table is now de-duplicated on its natural key and partitioned by `event_date` — this is what keeps queries over tables like `coldchain_sensor_readings` fast as they grow into millions of rows (Part 10 covers this further).

6.2 Curated table 1 — fleet_trip_telemetry (trips + GPS + vehicle master)

```

from pyspark.sql.functions import avg, max as smax, sum as ssum, count

gps_agg = (
    spark.table("workspace.silver_v2.fleet_gps_tracking")
    .groupBy("trip_id")
    .agg(
        avg("speed_kmh").alias("avg_speed_kmh"),
        smax("speed_kmh").alias("max_speed_kmh"),
        ssum("distance_traveled_km").alias("gps_distance_km"),
        count("*").alias("gps_ping_count"),
    )
)

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver_v2.fleet_trip_telemetry AS
SELECT
    t.trip_id, t.vehicle_id, t.corridor_name, t.start_location, t.end_location,
    t.planned_distance_km, t.actual_distance_km, t.planned_duration_hours, t.actual_duration_hours,
    t.start_time, t.end_time, t.fuel_consumed_liters, t.idle_time_minutes, t.delay_minutes,
    t.delay_reason,
    t.cargo_weight_tonnes, t.status,
    v.truck_type, v.max_load_tonnes, v.avg_consumption_l_per_100km, v.assigned_corridor,
    g.avg_speed_kmh, g.max_speed_kmh, g.gps_distance_km, g.gps_ping_count
FROM workspace.silver_v2.fleet_trips t
LEFT JOIN workspace.agl_core.dim_vehicle v ON t.vehicle_id = v.vehicle_id
LEFT JOIN (SELECT * FROM {gps_view}) g ON t.trip_id = g.trip_id
""").replace("{gps_view}", "gps_agg_view") # see note below
print("silver_v2.fleet_trip_telemetry ready.")

```

NOTE

`spark.sql` can't see a DataFrame directly — register it first: run `gps_agg.createOrReplaceTempView("gps_agg_view")` right before the `CREATE OR REPLACE TABLE` cell above (omitted inline to keep the SQL block readable). This is the same temp-view pattern v1 used for `staged_changes` in Part 4.4.

6.3 Curated table 2 — coldchain_compliance (sensor readings + container thresholds)

```

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver_v2.coldchain_compliance AS
SELECT
    r.reading_id, r.sensor_id, r.container_id, r.timestamp, r.event_date,
    r.temperature_c, r.humidity_percent, r.is_anomaly,
    c.product_type, c.setpoint_temp_c, c.temp_min_c, c.temp_max_c,
    c.setpoint_humidity_percent, c.humidity_min_percent, c.humidity_max_percent,

```

```

CASE WHEN r.temperature_c < c.temp_min_c OR r.temperature_c > c.temp_max_c THEN 1 ELSE 0 END AS
temp_excursion_flag,
CASE WHEN r.humidity_percent < c.humidity_min_percent OR r.humidity_percent > c.humidity_max_percent
THEN 1 ELSE 0 END AS humidity_excursion_flag
FROM workspace.silver_v2.coldchain_sensor_readings r
LEFT JOIN workspace.agl_core.dim_container c ON r.container_id = c.container_id
""")
print("silver_v2.coldchain_compliance ready.")

```

6.4 Curated table 3 — corridor_conditions_daily (weather + traffic, by corridor/day)

```

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver_v2.corridor_conditions_daily AS
SELECT
    w.corridor_name, w.event_date,
    avg(w.temperature_c) AS avg_temp_c, avg(w.humidity_percent) AS avg_humidity_pct,
    avg(w.precipitation_mm) AS avg_precip_mm, max(w.precipitation_mm) AS max_precip_mm,
    avg(tr.congestion_index) AS avg_congestion_index, avg(tr.average_speed_kmh) AS avg_road_speed_kmh,
    avg(tr.border_wait_time_hours) AS avg_border_wait_hours, sum(CASE WHEN tr.has_event THEN 1 ELSE 0
END) AS road_event_count
FROM workspace.silver_v2.ext_weather_data w
LEFT JOIN workspace.silver_v2.ext_traffic_data tr
    ON w.corridor_name = tr.corridor_name AND w.event_date = tr.event_date
GROUP BY w.corridor_name, w.event_date
""")
print("silver_v2.corridor_conditions_daily ready.")

```

6.5 Curated table 4 — driver_performance_daily (behavior + HOS + app usage, by driver/day)

```

spark.sql("""
CREATE OR REPLACE TABLE workspace.silver_v2.driver_performance_daily AS
SELECT
    b.driver_id, b.event_date,
    count(*) AS behavioral_event_count,
    sum(CASE WHEN b.event_type = 'Harsh Braking' THEN 1 ELSE 0 END) AS harsh_braking_count,
    sum(CASE WHEN b.severity = 'High' THEN 1 ELSE 0 END) AS high_severity_event_count,
    dl.overtime_minutes, dl.driving_minutes, dl.idle_minutes,
    app.avg_active_seconds
FROM workspace.silver_v2.driver_behavioral_events b
LEFT JOIN (
    SELECT driver_id, event_date,
        sum(CASE WHEN is_overtime THEN duration_minutes ELSE 0 END) AS overtime_minutes,
        sum(CASE WHEN duty_status = 'Driving' THEN duration_minutes ELSE 0 END) AS driving_minutes,
        sum(CASE WHEN duty_status = 'On Duty (Not Driving)' THEN duration_minutes ELSE 0 END) AS
idle_minutes
    FROM workspace.silver_v2.driver_logs GROUP BY driver_id, event_date
) dl ON b.driver_id = dl.driver_id AND b.event_date = dl.event_date
LEFT JOIN (
    SELECT driver_id, to_date(login_time) AS event_date, avg(app_active_time_seconds) AS
avg_active_seconds
    FROM workspace.silver_v2.driver_app_telematics GROUP BY driver_id, to_date(login_time)
) app ON b.driver_id = app.driver_id AND b.event_date = app.event_date
GROUP BY b.driver_id, b.event_date, dl.overtime_minutes, dl.driving_minutes, dl.idle_minutes,
app.avg_active_seconds
""")
print("silver_v2.driver_performance_daily ready.")

```

6.6 Curated table 5 — shipment_enriched (shipment + master data + delivery events)

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.silver_v2.shipment_enriched AS
SELECT
    s.*,
    cu.customer_type, cu.credit_limit,
    sh.shipper_name AS shipper_master_name,
    co.consignee_name AS consignee_master_name,
    dm.total_transit_hours, dm.total_transit_days, dm.on_time, dm.delay_hours,
    ex.exception_count
FROM workspace.silver_v2.shipment_data s
LEFT JOIN workspace.agl_core.dim_customer cu ON s.customer_id = cu.customer_id
LEFT JOIN workspace.agl_core.dim_shipper sh ON s.shipper_id = sh.shipper_id
LEFT JOIN workspace.agl_core.dim_consignee co ON s.consignee_id = co.consignee_id
LEFT JOIN workspace.silver_v2.delivery_metrics dm ON s.shipment_id = dm.shipment_id
LEFT JOIN (
    SELECT shipment_id, count(*) AS exception_count
    FROM workspace.silver_v2.delivery_exceptions GROUP BY shipment_id
) ex ON s.shipment_id = ex.shipment_id
""")
print("silver_v2.shipment_enriched ready.")
```

TIP

These five curated tables are exactly the join surface both personas need (Part 11). Building them once in Silver — instead of re-joining raw Bronze tables inside every Gold query — keeps Gold SQL short and keeps the expensive joins from running twice if two Gold tables need the same join.

PART 7

Notebook: 04_gold_feature_store_v2 — Data Scientist Tables

Builds the two feature tables the brief calls out: a Predictive ETA feature set (Fleet GPS + Weather + Historical Milestones) and a Spoilage Risk feature set (Environmental + Route Data).

7.1 fact_predictive_eta_features

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold_v2.fact_predictive_eta_features AS
SELECT
    se.shipment_id, se.customer_id, se.customer_type, se.container_type, se.weight_kg, se.volume_m3,
    se.guaranteed_transit_days, se.order_date, se.dispatch_date, se.delivery_deadline,
se.actual_delivery_date,
    se.total_transit_hours, se.total_transit_days, se.on_time, se.delay_hours,
    ft.trip_id, ft.corridor_name, ft.planned_distance_km, ft.actual_distance_km,
    ft.planned_duration_hours, ft.actual_duration_hours, ft.avg_speed_kmh, ft.max_speed_kmh,
    ft.idle_time_minutes, ft.delay_minutes AS trip_delay_minutes, ft.delay_reason AS trip_delay_reason,
    cc.avg_temp_c, cc.avg_precip_mm, cc.max_precip_mm, cc.avg_congestion_index,
    cc.avg_road_speed_kmh, cc.avg_border_wait_hours, cc.road_event_count,
    ms_agg.milestone_count, ms_agg.customs_hold_flag
FROM workspace.silver_v2.shipment_enriched se
LEFT JOIN workspace.silver_v2.fleet_trip_telemetry ft
    ON se.shipment_id = ft.trip_id -- adjust join key once trip<->shipment mapping is confirmed in your
data
LEFT JOIN workspace.silver_v2.corridor_conditions_daily cc
    ON ft.corridor_name = cc.corridor_name AND to_date(se.dispatch_date) = cc.event_date
LEFT JOIN (
    SELECT shipment_id, count(*) AS milestone_count,
        max(CASE WHEN milestone_name = 'Customs Cleared' THEN 1 ELSE 0 END) AS customs_hold_flag
    FROM workspace.silver_v2.delivery_milestones GROUP BY shipment_id
) ms_agg ON se.shipment_id = ms_agg.shipment_id
""")
print("gold_v2.fact_predictive_eta_features ready – label columns: total_transit_hours / on_time")
```

WATCH OUT

Your generators produce `shipment_data` and `fleet_trips` independently, so there may be no direct `shipment_id` → `trip_id` key yet. Before running this in production, add (or confirm) a linking column — either a `shipment_id` column on `trips`, or a bridge table. Until then, join on `corridor_name` + nearest date as an approximation, or build the ETA feature table at the trip grain instead of the shipment grain.

7.2 fact_spoilage_risk_features

```
spark.sql("""
CREATE OR REPLACE TABLE workspace.gold_v2.fact_spoilage_risk_features AS
SELECT
    cc.container_id, cc.event_date, cc.product_type,
    count(*) AS reading_count,
    sum(cc.temp_excursion_flag) AS temp_excursion_count,
    sum(cc.humidity_excursion_flag) AS humidity_excursion_count,
    avg(cc.temperature_c) AS avg_temp_c, min(cc.temperature_c) AS min_temp_c, max(cc.temperature_c) AS
```

```

max_temp_c,
  cd.avg_temp_c AS ambient_avg_temp_c, cd.avg_precip_mm AS ambient_avg_precip_mm,
  rs.compressor_cycles_per_hour, rs.defrost_cycle_count, rs.power_status,
  CASE
    WHEN sum(cc.temp_excursion_flag) > 5 THEN 'High'
    WHEN sum(cc.temp_excursion_flag) BETWEEN 1 AND 5 THEN 'Medium'
    ELSE 'Low'
  END AS spoilage_risk_label
FROM workspace.silver_v2.coldchain_compliance cc
LEFT JOIN workspace.silver_v2.corridor_conditions_daily cd
  ON cd.event_date = cc.event_date
LEFT JOIN workspace.silver_v2.coldchain_reefer_status rs
  ON cc.container_id = rs.container_id AND cc.event_date = to_date(rs.timestamp)
GROUP BY cc.container_id, cc.event_date, cc.product_type, cd.avg_temp_c, cd.avg_precip_mm,
         rs.compressor_cycles_per_hour, rs.defrost_cycle_count, rs.power_status
""")
print("gold_v2.fact_spoilage_risk_features ready – label column: spoilage_risk_label")

```

Expected result: both tables created; row counts printed via `spark.table(...).count()` if you add a verification cell, matching v1 Part 2.6's habit of always confirming row counts after a build.

Data Scientists: both tables are already at a clean training grain (one row per shipment, or one row per container-day) — read them straight into a notebook with `spark.table("workspace.gold_v2.fact_predictive_eta_features").toPandas()` and go straight to feature scaling / train-test split.

PART 8

Notebook: 05_gold_business_views_v2 — Business Leader Views

Builds Cost-per-Kilometer and profitability-leak views from Financial + Driver App + Telematics data, as views (not materialized tables) so they always reflect the latest Silver refresh without a rebuild step — matching v1 Part 8's `CREATE OR REPLACE VIEW` pattern.

8.1 vw_cost_per_km

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold_v2.vw_cost_per_km AS
WITH fuel_cost AS (
    SELECT ft.trip_id, ft.vehicle_id, ft.actual_distance_km, ft.fuel_consumed_liters,
           ft.fuel_consumed_liters * fp.fuel_price_usd AS fuel_cost_usd
    FROM workspace.silver_v2.fleet_trip_telemetry ft
    LEFT JOIN (
        SELECT to_date(timestamp) AS price_date, avg(fuel_price_usd) AS fuel_price_usd
        FROM workspace.silver_v2.ext_fuel_prices WHERE country = 'Rwanda' GROUP BY to_date(timestamp)
    ) fp ON to_date(ft.start_time) = fp.price_date
),
maintenance_cost AS (
    SELECT vehicle_id, sum(cost) / nullif(sum(coalesce(duration_hours,0)), 0) AS maint_cost_per_hour
    FROM workspace.silver_v2.fleet_maintenance_logs GROUP BY vehicle_id
)
SELECT
    fc.trip_id, fc.vehicle_id, fc.actual_distance_km, fc.fuel_consumed_liters, fc.fuel_cost_usd,
    dp.overtime_minutes, dp.idle_minutes,
    (dp.overtime_minutes / 60.0) * 15.0 AS overtime_cost_usd,      -- USD 15/hr assumption, replace with
    your real driver rate
    mc.maint_cost_per_hour,
    (fc.fuel_cost_usd
     + coalesce((dp.overtime_minutes / 60.0) * 15.0, 0)
     + coalesce(mc.maint_cost_per_hour, 0)
    ) / nullif(fc.actual_distance_km, 0) AS cost_per_km_usd
FROM fuel_cost fc
LEFT JOIN workspace.silver_v2.driver_performance_daily dp
    ON fc.vehicle_id = dp.driver_id  -- adjust to your real vehicle<->driver_daily join key
LEFT JOIN maintenance_cost mc ON fc.vehicle_id = mc.vehicle_id
""")
print("gold_v2.vw_cost_per_km ready.")
```

NOTE

The overtime driver-rate constant and the `vehicle_id ↔ driver_id` join are placeholders — plug in your real driver pay rate and the actual vehicle-to-driver assignment key (available on `dim_vehicle.assigned_driver`) once you finalize the linkage between trips and driver-day performance.

8.2 vw_fleet_idle_leakage — where profit is leaking

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold_v2.vw_fleet_idle_leakage AS
SELECT
    ft.vehicle_id, ft.corridor_name,
    count(*) AS trip_count,
    avg(ft.idle_time_minutes) AS avg_idle_minutes,
    avg(ft.actual_duration_hours - ft.planned_duration_hours) AS avg_overrun_hours,
    avg(ft.fuel_consumed_liters / nullif(ft.actual_distance_km,0) * 100) AS avg_l_per_100km,
    v.avg_consumption_l_per_100km AS rated_l_per_100km,
    avg(ft.fuel_consumed_liters / nullif(ft.actual_distance_km,0) * 100) - v.avg_consumption_l_per_100km
AS excess_l_per_100km
FROM workspace.silver_v2.fleet_trip_telemetry ft
LEFT JOIN workspace.agl_core.dim_vehicle v ON ft.vehicle_id = v.vehicle_id
GROUP BY ft.vehicle_id, ft.corridor_name, v.avg_consumption_l_per_100km
ORDER BY excess_l_per_100km DESC
""")
print("gold_v2.vw_fleet_idle_leakage ready – sorted worst-offenders first.")
```

8.3 vw_otif_performance — On-Time-In-Full, by customer and corridor

```
spark.sql("""
CREATE OR REPLACE VIEW workspace.gold_v2.vw_otif_performance AS
SELECT
    se.customer_id, se.customer_type, count(*) AS shipment_count,
    sum(CASE WHEN se.on_time THEN 1 ELSE 0 END) AS on_time_count,
    round(sum(CASE WHEN se.on_time THEN 1 ELSE 0 END) / count(*) * 100, 1) AS otif_percent,
    avg(se.total_cost) AS avg_total_cost, sum(se.total_cost) AS total_revenue_at_risk
FROM workspace.silver_v2.shipment_enriched se
GROUP BY se.customer_id, se.customer_type
ORDER BY otif_percent ASC
""")
print("gold_v2.vw_otif_performance ready.")
```

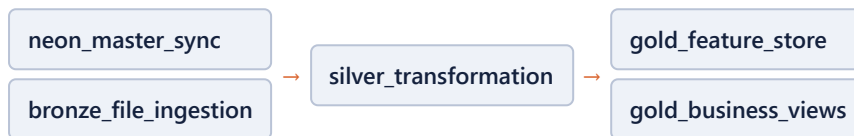
Expected result: three views created, queryable directly from a BI tool (Databricks SQL, Power BI, Tableau) via Databricks SQL Warehouse — no notebook needed to consume them day to day.

PART 9

Orchestrate: the AGL_ingestion_v2 Job

9.1 Why the task graph branches (unlike v1's straight chain)

v1's job was a single chain: sync → bronze → silver → gold. v2 has two independent ingestion paths (Neon sync and file ingestion don't depend on each other) that both must finish before Silver can join across them, and two independent Gold notebooks that can build side by side once Silver is ready:



9.2 Build it in Workflows

1. Workflows → **Create Job** → rename to `AGL_ingestion_v2`.
2. **Task 1** — name `neon_master_sync`, notebook `01_neon_master_sync_v2`, no dependency.
3. **Task 2** — name `bronze_file_ingestion`, notebook `02_bronze_file_ingestion_v2`, no dependency (runs in parallel with Task 1).
4. **Task 3** — name `silver_transformation`, notebook `03_silver_transformation_v2`, **Depends on:** check both `neon_master_sync` and `bronze_file_ingestion`.
5. **Task 4** — name `gold_feature_store`, notebook `04_gold_feature_store_v2`, **Depends on:** `silver_transformation`.
6. **Task 5** — name `gold_business_views`, notebook `05_gold_business_views_v2`, **Depends on:** `silver_transformation` (runs in parallel with Task 4).

Expected result: the job graph shows a diamond — two parallel starting tasks converging into `silver_transformation`, which then fans back out into two parallel Gold tasks. Total run time is roughly $\max(\text{Neon sync, file ingestion}) + \text{silver} + \max(\text{feature store, business views})$ — noticeably faster than running all five serially.

TIP

Use a Job Cluster (v1 Part 4.2's tip) sized for the file-ingestion task specifically — it's the heaviest (Parquet/JSON streaming reads across 22 sources). A single shared cluster across all 5 tasks is simplest to start with; split into per-task clusters later only if you see queueing.

9.3 Notifications

Same as v1 Part 5.2 — add **On failure** at the job level so any of the 5 tasks failing pages you. Since Task 3 depends on both Task 1 and Task 2, a failure in either one correctly shows Tasks 4/5 as "Upstream failed" rather than running Gold against half-synced data — the same safety property v1 Part 6 documents.

PART 10

Schedule, Scale & Performance

10.1 Schedule

Fleet GPS, telematics and driver-behavior data arrive far more frequently than the beginner guide's nightly batch warrants. Recommended cadence:

Trigger	Cadence	Rationale
Scheduled	Every 6 hours	Balances freshness for BI/ML against cluster cost — good default to start with
File arrival	On new file in <code>agl_volume_v2/</code>	Best fit for the telemetry-heavy domains (fleet, coldchain, driver) if you want near-real-time Gold refresh

Set it the same way as v1 Part 5.1, pointed at `/Volumes/workspace/agl_core/agl_volume_v2/` if you choose the file-arrival option.

10.2 Partitioning & file layout

Every Silver table in Part 6.1 is already written with `partitionBy("event_date")` — this is what lets a query like "last 7 days of GPS pings" skip reading the other 99% of the table as the dataset grows from thousands to millions of rows. For the three highest-volume tables specifically (`coldchain_sensor_readings` , `fleet_gps_tracking` , `driver_logs`), also enable Delta's Liquid Clustering on the natural join key so downstream joins (to `dim_container` , `dim_vehicle` , `dim_driver`) stay fast without manual `OPTIMIZE ... ZORDER BY` maintenance:

```
ALTER TABLE workspace.silver_v2.coldchain_sensor_readings CLUSTER BY (container_id, event_date);
ALTER TABLE workspace.silver_v2.fleet_gps_tracking          CLUSTER BY (vehicle_id, event_date);
ALTER TABLE workspace.silver_v2.driver_logs                 CLUSTER BY (driver_id, event_date);

-- run periodically (or as a 6th job task after silver_transformation)
OPTIMIZE workspace.silver_v2.coldchain_sensor_readings;
OPTIMIZE workspace.silver_v2.fleet_gps_tracking;
OPTIMIZE workspace.silver_v2.driver_logs;
```

10.3 Schema drift & quarantine

v1 Part 6 shows the "cloudFiles: schema location already contains a schema and it doesn't match" error and its fix (delete the schema-location folder, let it re-infer). At v2's scale across 22 Auto Loader sources, don't wait for that error — build it in from day one via `cloudFiles.schemaEvolutionMode = "addNewColumns"` (already set in Part 5.2), which lets a source add columns without failing the stream. For rows that fail type casts entirely, add a quarantine sink per source using the same pattern v1's warehouse-inventory troubleshooting introduced — write rows that fail a business-rule check (e.g. negative `weight_kg` , a `shipment_id` that doesn't match the expected `SHIP_#####` pattern) to a sibling `*_quarantine` table instead of silently dropping or silently failing the batch.

10.4 Cluster sizing

All 31 tables here are still small-to-medium by Spark standards (low thousands to tens of thousands of rows per table today). The architecture is written to scale linearly as your generators produce more days of data — Auto Loader and partitioned Delta tables don't need to change as row counts grow into the millions, only the cluster's worker count does. Start with a small autoscaling Job Cluster (2–8 workers) and watch the Spark UI's task-skew metrics on the `bronze_file_ingestion` task before sizing up.

PART 11

How Each Persona Actually Uses This

11.1 Data Scientist — Predictive ETA model

Source: `gold_v2.fact_predictive_eta_features` (built in Part 7.1 from Fleet GPS + Weather + Historical Milestones).

```
eta_df = spark.table("workspace.gold_v2.fact_predictive_eta_features").toPandas()

features = [
    "guaranteed_transit_days", "weight_kg", "volume_m3", "planned_distance_km",
    "avg_speed_kmh", "idle_time_minutes", "avg_temp_c", "avg_precip_mm",
    "avg_congestion_index", "avg_border_wait_hours", "milestone_count",
]
label = "total_transit_hours" # or `on_time` for a classification model

from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor

X_train, X_test, y_train, y_test = train_test_split(
    eta_df[features].fillna(0), eta_df[label], test_size=0.2, random_state=42
)
model = GradientBoostingRegressor().fit(X_train, y_train)
print("R^2 on holdout:", model.score(X_test, y_test))
```

11.2 Data Scientist — Spoilage Risk model

Source: `gold_v2.fact_spoilage_risk_features` (built in Part 7.2 from Environmental + Route Data).

```
risk_df = spark.table("workspace.gold_v2.fact_spoilage_risk_features").toPandas()

features = [
    "reading_count", "avg_temp_c", "min_temp_c", "max_temp_c",
    "ambient_avg_temp_c", "ambient_avg_precip_mm", "compressor_cycles_per_hour", "defrost_cycle_count",
]
from sklearn.ensemble import RandomForestClassifier
X = risk_df[features].fillna(0)
y = risk_df["spoilage_risk_label"]
clf = RandomForestClassifier(n_estimators=200, random_state=42).fit(X, y)
print("Feature importances:", dict(zip(features, clf.feature_importances_)))
```

11.3 Business Leader — Cost-per-Kilometer & profit leaks

Source: `gold_v2.vw_cost_per_km` and `gold_v2.vw_fleet_idle_leakage` (Part 8), queried straight from a Databricks SQL Warehouse or a connected BI tool — no Python required:

```
-- Which vehicles are burning the most money per km?
SELECT vehicle_id, round(avg(cost_per_km_usd), 3) AS avg_cost_per_km_usd, count(*) AS trips
FROM workspace.gold_v2.vw_cost_per_km
```

```
GROUP BY vehicle_id
ORDER BY avg_cost_per_km_usd DESC
LIMIT 20;

-- Where is idling/inefficiency leaking profit?
SELECT vehicle_id, corridor_name, avg_idle_minutes, excess_l_per_100km
FROM workspace.gold_v2.vw_fleet_idle_leakage
WHERE excess_l_per_100km > 0
ORDER BY excess_l_per_100km DESC
LIMIT 20;
```

This directly answers the brief: "identify leaking profits from excessive vehicle idling or inefficient routes" — the worst offenders sort to the top of both queries.

PART 12

Troubleshooting (v2-specific)

Symptom	Fix
One source in the Part 5.2 loop prints "FAILED" but the rest still ran	That's by design (the try/except). Check the printed exception, fix that one source's format/glob/schema, and re-run — the notebook is idempotent thanks to Auto Loader's own checkpoints, so re-running doesn't double-ingest the sources that already succeeded.
<code>to_sql</code> in Part 1.3 fails with a duplicate key error	Seeding already happened. Don't re-run against a populated table; if you must reseed, <code>TRUNCATE</code> the 8 Neon tables first.
Silver curated tables in Part 6.2–6.6 return mostly-NULL joined columns	Almost always a join-key mismatch — your generators created each domain's IDs independently, so a <code>trip_id</code> -to- <code>shipment_id</code> or <code>vehicle_id</code> -to- <code>driver_id</code> bridge may not exist yet. Confirm the real linking key in your data before trusting the join (flagged inline in Parts 7.1 and 8.1).
Gold Job tasks (Part 9) show "Upstream failed" for both parallel branches	Expected if <code>silver_transformation</code> failed — same skip-on-failure behavior v1 Part 6 documents, just now with two upstream tasks instead of one.
Parquet-based Auto Loader source (gps_tracking, telematics, sensor_readings, reefer_status) infers a column as the wrong type	Parquet carries its own schema, so this is rarer than with CSV/JSON — if it happens, add a <code>cloudFiles.schemaHints</code> option the same way v1 Part 6 fixed <code>Pallet_Count</code> .

PART 13

Glossary Additions

Term	Meaning
Config-driven ingestion	Describing each data source as one row in a list/table instead of one hand-written code cell per source, so one function/loop handles all of them.
Curated Silver table	A Silver-layer table that joins two or more domains together (e.g. trips + GPS + vehicle master) so Gold tables don't have to repeat the join.
Feature table	A Gold table shaped specifically for model training — one row per prediction unit (shipment, or container-day), with the label column included.
Liquid Clustering	A Delta table maintenance mode (<code>CLUSTER BY</code>) that keeps data co-located by key without needing manual periodic <code>OPTIMIZE ... ZORDER BY</code> runs.
Schema evolution mode	The Auto Loader option controlling what happens when a new column shows up in a source file — <code>addNewColumns</code> accepts it instead of failing the stream.
Diamond-shaped job graph	Two tasks with no dependency on each other, both feeding into one downstream task, which then fans back out into two more independent downstream tasks — lets independent work run in parallel.

AGL Lakehouse v2 — Enterprise-Scale Orchestration Guide. Generated for the AGL Rwanda data engineering project.